Task 1: Generating Two Different Files with the Same MD5 Hash

Hash files into binary format

```
seed@cisc468:~/a3$ ./md5collgen -p pre.dat -o out1.bin out2.bin
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'out1.bin' and 'out2.bin'
Using prefixfile: 'pre.dat'
Using initial value: 1d9a946f4bcc62c40488a5b61ca36f7d

Generating first block: ........
Generating second block: W.
Running time: 7.38657 s
seed@cisc468:~/a3$ diff out1.bin out2.bin
Binary files out1.bin and out2.bin differ
seed@cisc468:~/a3$ md5sum out1.bin
b5b61f5bd3ef9e8d193adb4c766ddfad  out1.bin
seed@cisc468:~/a3$ md5sum out2.bin
b5b61f5bd3ef9e8d193adb4c766ddfad  out2.bin
```

My content of pre.dat is "Arbitrary content from Kevin Yao"

```
d@cisc468:~/a3$ xxd out1.bin
00000000: 4172 6269 7472 6172 7920 636f 6e74 656e  Arbitrary conten
00000010: 7420 6672 6f6d 204b 6576 696e 2059 616f  t from Kevin Yao
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000040: 2ec4 bace 8dab 3ac2 e5b2 1f9e 1d54 c9ef  ......:......T..
00000050: da6b 8e31 797c 3e93 2f72 79ae 7198 b632  .k.1y|>./ry.q..2
00000060: 24e6 aac8 4865 0c00 e743 cd9a ef17 36fe  $...He...C....6.
00000070: df1a 5505 8baf 7d25 c0c9 a38f 33e7 8a65  ..U...}%....3..e
00000080: d471 8d34 2c95 d3ea 9d0f 5c34 9c12 6b1b  .q.4,.....\4..k.
00000090: 1795 9b26 4738 352c facc f326 d15c fdad  ...&G85,...&.\..
000000a0: 1d40 be2a 3798 cc68 d617 017a 9874 49bd  .@.*7..h...z.tI.
000000b0: d5b4 7c01 aec4 a3b0 bba1 10d5 626a d26a  ..|.........bj.j
```

```
seed@cisc468:~/a3$ xxd out2.bin
00000000: 4172 6269 7472 6172 7920 636f 6e74 656e  Arbitrary conten
00000010: 7420 6672 6f6d 204b 6576 696e 2059 616f  t from Kevin Yao
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000040: 2ec4 bace 8dab 3ac2 e5b2 1f9e 1d54 c9ef  ......:......T..
00000050: da6b 8eb1 797c 3e93 2f72 79ae 7198 b632  .k..y|>./ry.q..2
00000060: 24e6 aac8 4865 0c00 e743 cd9a ef97 36fe  $...He...C....6.
00000070: df1a 5505 8baf 7d25 c0c9 a30f 33e7 8a65  ..U...}%....3..e
00000080: d471 8d34 2c95 d3ea 9d0f 5c34 9c12 6b1b  .q.4,.....\4..k.
00000090: 1795 9ba6 4738 352c facc f326 d15c fdad  ....G85,...&.\..
000000a0: 1d40 be2a 3798 cc68 d617 017a 98f4 48bd  .@.*7..h...z..H.
000000b0: d5b4 7c01 aec4 a3b0 bba1 1055 626a d26a  ..|........Ubj.j
```

xxd is an alternative to `bless` that shows the content of a binary file.

1.

If the prefix file length is not a multiple of 64 bytes, md5collgen will automatically pad it with zeros until it reaches the next multiple of 64 bytes. As we can see in the above output. The first 64 hex digits (32 bytes) have content, the remaining 32 bytes are padded zeroes.
This is because MD5 operates on 64-byte (512-bit) blocks. The padding is inserted between the original prefix and the 128 bytes of collision data generated by the tool. Both output files contain the same padded prefix followed by their respective 128-byte collision blocks.

2.

```
seed@cisc468:~/a3$ ./md5collgen -p pre.dat -o out1.bin out2.bin
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'out1.bin' and 'out2.bin'
Using prefixfile: 'pre.dat'
Using initial value: 6b08ea70735a3ba70c27eab6324df08d

Generating first block: ..................................................
..........................................................................
Generating second block: S01...
Running time: 181.898 s
seed@cisc468:~/a3$ xxd out1.bin
00000000: 4172 6269 7472 6172 7920 636f 6e74 656e  Arbitrary conten
00000010: 7420 6672 6f6d 204b 6576 696e 2059 616f  t from Kevin Yao
00000020: 4172 6269 7472 6172 7920 636f 6e74 656e  Arbitrary conten
00000030: 7420 6672 6f6d 204b 6576 696e 2059 616f  t from Kevin Yao
00000040: 2a69 4bd2 7b64 4735 c277 1e04 4836 d1b0  *iK.{dG5.w..H6..
00000050: f63f 0e77 1648 ef26 0e16 44ba 1026 afca  .?.w.H.&..D..&..
00000060: 41c4 bee9 caae 1b33 8b20 6d11 d221 1644  A......3. m..!.D
00000070: c8e9 f2a5 a29f 0401 f15f 2f86 4ac9 d05e  .........._/.J..^
00000080: fc1c 996d 5dfe f89b ef76 1bd0 d3af f1e2  ...m]....v......
00000090: 0a9e b109 0f06 26e1 cb1a 6e39 ec87 dee5  ......&...n9....
000000a0: b08d c69a f6d7 dffc d84b 7457 aff8 616e  .........KtW..an
000000b0: 61a3 4058 fd99 7658 ff25 1337 ccda 7c37  a.@X..vX.%.7..|7
```

When the prefix file is exactly 64 bytes (one full MD5 block), no padding is needed. The tool uses the prefix as-is and appends the 128 bytes of generated collision data directly after the prefix.
As you can see, the pre.dat now contains the same content copied twice (perfectly 64 bytes)
Each output file will be exactly 192 bytes (64 prefix + 128 collision data).

3.
In the first version (with padded zeroes) we can see that most of the collision bits are actually very similar.

```
1  4172 6269 7472 6172 7920 636f 6e74 656e
2  00000010: 7420 6672 6f6d 204b 6576 696e 2059 616f
3  00000040: 2ec4 bace 8dab 3ac2 e5b2 1f9e 1d54 c9ef
4  00000050: da6b 8e31 797c 3e93 2f72 79ae 7198 b632
5  00000060: 24e6 aac8 4865 0c00 e743 cd9a ef17 36fe
6  00000070: dfla 5505 8baf 7d25 c0c9 a38f 33e7 8a65
7  00000080: d471 8d34 2c95 d3ea 9d0f 5c34 9c12 6b1b
8  00000090: 1795 9b26 4738 352c facc f326 d15c fdad
9  000000a0: 1d40 be2a 3798 cc68 d617 017a 9874 49bd
10 000000b0: d5b4 7c01 aec4 a3b0 bbal 10d5 626a d26a
```

```
1  4172 6269 7472 6172 7920 636f 6e74 656e
2  00000010: 7420 6672 6f6d 204b 6576 696e 2059 616f
3  00000040: 2ec4 bace 8dab 3ac2 e5b2 1f9e 1d54 c9ef
4  00000050: da6b 8eb1 797c 3e93 2f72 79ae 7198 b632
5  00000060: 24e6 aac8 4865 0c00 e743 cd9a ef97 36fe
6  00000070: dfla 5505 8baf 7d25 c0c9 a30f 33e7 8a65
7  00000080: d471 8d34 2c95 d3ea 9d0f 5c34 9c12 6b1b
8  00000090: 1795 9ba6 4738 352c facc f326 d15c fdad
9  000000a0: 1d40 be2a 3798 cc68 d617 017a 98f4 48bd
10 000000b0: d5b4 7c01 aec4 a3b0 bbal 1055 626a d26a
```

The difference is shown here. 7 of the 128 bytes of collision are different, the rest are the same.

Task 2: Understanding MD5

I've split this task into multiple tests that tests adding strings, or random bytes separately.

Test: Empty String

I would begin to test this property with an empty string.
I create suffix.txt, which contains just the empty string (nothing). Then I run these commands

```
seed@cisc468:~/a3$ cat out1.bin suffix.txt > combined1.bin
seed@cisc468:~/a3$ cat out2.bin suffix.txt > combined2.bin
seed@cisc468:~/a3$ md5sum combined1.bin combined2.bin
5056aa28ce09a37af1d982cae4887afe  combined1.bin
5056aa28ce09a37af1d982cae4887afe  combined2.bin
```

The command shows that both have the same hash,

Test: "Hello World"

Now I edit suffix.txt to contain "Hello World" text, as a form of real message appended

```
seed@cisc468:~/a3$ cat out1.bin suffix.txt > combined1_wtext.bin
seed@cisc468:~/a3$ cat out2.bin suffix.txt > combined2_wtext.bin
seed@cisc468:~/a3$ md5sum combined1_wtext.bin combined2_wtext.bin
32382dbf632983af69c5648e41a6fd08  combined1_wtext.bin
32382dbf632983af69c5648e41a6fd08  combined2_wtext.bin
```

The md5sum command once again shows that they have the same hash.

Test: Random 16 Bytes

Now I run: head -c 16 /dev/urandom, to see if the property holds on random data

```
seed@cisc468:~/a3$ head -c 16 /dev/urandom > suffix_t3.bin
seed@cisc468:~/a3$ cat out1.bin suffix_t3.bin > concat1_t3.bin
seed@cisc468:~/a3$ cat out2.bin suffix_t3.bin > concat2_t3.bin
seed@cisc468:~/a3$ md5sum concat1_t3.bin concat2_t3.bin
931b40ec0524fd75aa7109914bab5b21  concat1_t3.bin
931b40ec0524fd75aa7109914bab5b21  concat2_t3.bin
```

We can see that they still have the same hash

Test: Random 256 Bytes

The purpose of this test is to check if the property holds even when we have random data that spans multiple blocks for MD5.

```
seed@cisc468:~/a3$ head -c 256 /dev/urandom > suffix_t4.bin
seed@cisc468:~/a3$ cat out1.bin suffix_t4.bin > concat1_t4.bin
seed@cisc468:~/a3$ cat out2.bin suffix_t4.bin > concat2_t4.bin
seed@cisc468:~/a3$ md5sum concat1_t4.bin concat2_t4.bin
8a6ef970d0fb346de6dea3e385176b64  concat1_t4.bin
8a6ef970d0fb346de6dea3e385176b64  concat2_t4.bin
```

It appears so as well! Looks like appending text/bytes and md5 still outputs the same hash always

The reason that the above property holds is mostly because: once two messages reach the same intermediate hash value after processing their existing blocks, feeding the same suffix blocks into the same compression function produces the same final digest/exectuable/hash as well. This is a flaw of cryptographic functions processing each block sequentially.

Task 3: Generating Two Executable Files with the Same MD5 Hash

Here's the C code

```c
#include <stdio.h>
unsigned char xyz[200] = {
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41
};

int main() {
   int i;
   for (i=0; i<200; i++) {
       printf("%x", xyz[i]);
   }
   printf("\n");
   return 0;
}
```

Essentially, we follow the instructions and put 200 bytes of the character 'A' (0x41) into the array.

This allows us to have a block of content in the binary executable that we can replace for the rest of this task.

Because this part is 200 bytes, it is guaranteed that we can find a point where one A ends at a location that is a multiple of 64 bytes (end of one block), and have at least 128 bytes remaining that we can safely replace to content of our choice, and have As remaining for the next block

We now generate our executable and locate the location of the As

```
seed@cisc468:~/a3$ gcc -o task3 task3.c
seed@cisc468:~/a3$ xxd task3 | grep -n "4141"
771:00003020: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
772:00003030: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
773:00003040: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
774:00003050: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
775:00003060: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
776:00003070: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
777:00003080: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
778:00003090: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
779:000030a0: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
780:000030b0: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
781:000030c0: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
782:000030d0: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
783:000030e0: 4141 4141 4141 4141 4743 433a 2028 5562  AAAAAAAAGCC: (Ub
seed@cisc468:~/a3$ grep -boa $'\x41\x41\x41\x41\x41\x41\x41\x41' task3 | head -1
12320:AAAAAAAA
```

The red part shows there are indeed 200 consecutive bytes of As in the executable

The command grep -boa prints b for byte offset, o for the matching part, and a for treating this binary file as plaintext so the search could work, then head -1 shows only the first match.

Seems like byte 12320 is where our As begin, but 12320 is not a perfect multiple of 64. The closest match is byte 12352/64 = 193. So our offset should be 12352.

Now make a bash file that executes the experiment

```bash
#!/bin/bash

OFFSET=12352

# Split the binary into prefix and suffix
head -c $OFFSET task3 > prefix.bin
tail -c +$((OFFSET + 129)) task3 > suffix.bin

# Generate two collision blocks from the prefix
./md5collgen -p prefix.bin -o col1.bin col2.bin

# Append the suffix to both collision outputs
cat col1.bin suffix.bin > prog1
```

```
cat col2.bin suffix.bin > prog2

# Make both executable
chmod +x prog1 prog2

# Verify same hash
echo "MD5 hashes:"
md5sum prog1 prog2

# Verify different output
echo "Program 1 output:"
./prog1
echo "Program 2 output:"
./prog2
```

We start by splitting the binary into prefix and suffix binary files, using the OFFSET we've found.

Then we generate 2 collision blocks using prefix.
Then we append the suffix (everything after the next 128 bytes of A, meaning some more As and the rest of the executable file)

Finally, we combine the two parts of the executable, but with the hash collision blocks appearing after prefix.
Now we execute both programs and see the output:

```
seed@cisc468:~/a3$ ./task3.sh
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'col1.bin' and 'col2.bin'
Using prefixfile: 'prefix.bin'
Using initial value: 786e9083e0ffc70227f3918917ffca66

Generating first block: ...
Generating second block: W....
Running time: 0.801104 s
MD5 hashes:
01ba373ba5346f5a2510c7e519779c19  prog1
01ba373ba5346f5a2510c7e519779c19  prog2

Program 1 output:
414141414141414141414141414141414141414141414141414141414141414141414141ee47f3748aee316f1fead8582f6892dcf12ecb8f8c4d7c248e4185
12aace0531ee1d1ee4cde594b304711915dc1597be4ebb674f9aeeddb3489525d8d6b675c35a07945c734ba358d97b8b798a6dfe9ab6a7341783ed
e8a869a84d7ced9e1510b39afc8f89161a459a95f0351ddfc8dff052cac398add3ce6e7e2b851c414141414141414141414141414141414141414141
414141414141414141414141414141414141414141
Program 2 output:
414141414141414141414141414141414141414141414141414141414141414141414141ee47f3748aee316f1fead8582f6892dcf12ec38f8c4d7c248e4185
12aace0531ee1d1ee4cde594b30471191ddc1597be4ebb674f9aeeddb3489d25d8d6b675c35a07945c734ba358d97b8b798a6dfe9ab627341783ed
e8a869a84d7ced9e1510b39afc8f89161a459a9570351ddfc8dff052cac398add3ceee7e2b851c414141414141414141414141414141414141414141
414141414141414141414141414141414141414141
```

We can see the two programs indeed have the same hash, but

```
Program 1 output:
414141414141414141414141414141414141414141414141414141414141414141414141ee47f3748aee316f1fead858
2f6892dcf12ecb8f8c4d7c248e418512aace0531ee1d1ee4cde594b304711915dc1597be4ebb674f9aeeddb3
489525d8d6b675c35a07945c734ba358d97b8b798a6dfe9ab6a7341783ede8a869a84d7ced9e1510b39afc8f
89161a459a95f0351ddfc8dff052cac398add3ce6e7e2b851c414141414141414141414141414141414141414141
414141414141414141414141414141414141414141

Program 2 output:
414141414141414141414141414141414141414141414141414141414141414141414141ee47f3748aee316f1fead858
2f6892dcf12ec38f8c4d7c248e418512aace0531ee1d1ee4cde594b304711191ddc1597be4ebb674f9aeeddb3
489d25d8d6b675c35a07945c734ba358d97b8b798a6dfe9ab627341783ede8a869a84d7ced9e1510b39afc8f
89161a459a9570351ddfc8dff052cac398add3ceee7e2b851c414141414141414141414141414141414141414141
414141414141414141414141414141414141414141
```

Their program output (which prints the content of the xyz array we initialized) differs by a few collision bytes after replacing the middle 128 bytes of As.

This attack works because we knew the content of the executable beforehand, that the space of xyz is free to manipulate and will not break execution since we can put virtually any byte inside the byte array.

Task 4: Making the Two Programs Behave Differently

To achieve this, we follow the example and establish two arrays, X and Y, that are both 200 bytes, filled with As. We also have an if statement that compares them: if X and Y are identical, run benign code; otherwise run malicious code. We then compile this into a binary called task4.

```
#include <stdio.h>
#include <string.h>

unsigned char X[200] = {
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41
};

unsigned char Y[200] = {
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
   0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
```

```
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41,
    0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41
};

int main() {
    if (memcmp(X, Y, sizeof(X)) == 0) {
        printf("Benign: running safe functionality.\n");
    } else {
        printf("Malicious: running dangerous functionality! >:)\n");
    }
    return 0;
}
```

What we need to do now is, we locate the first occurrence of the A's block in the binary to find where Array X lives, and align that offset down to a multiple of 64 to get our prefix split point. We also locate the second occurrence of the A's block to find where Array Y lives.

This gives us four regions to extract: the prefix (everything before X's aligned start), the middle (everything between the end of X's region and the start of Y), the suffix (everything after Y's 128-byte region), and the two collision blocks P and Q generated by running md5collgen on the prefix.

By printing the locations of where 4141 is found (see next page), we can see that X starts at byte 12320, and Y starts at byte 12544.
The closest match for a perfect 128 block is: 12352 for X, which consequently, means we must use 12352 + 128 + 64 = 12544 as the start of Y so each block stays perfectly 64 bytes.

```
seed@cisc468:~/a3$ xxd task4 | grep -n "4141"
771:00003020: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
772:00003030: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
773:00003040: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
774:00003050: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
775:00003060: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
776:00003070: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
777:00003080: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
778:00003090: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
779:000030a0: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
780:000030b0: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
781:000030c0: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
782:000030d0: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
783:000030e0: 4141 4141 4141 4141 0000 0000 0000 0000  AAAAAAAA........
785:00003100: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
786:00003110: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
787:00003120: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
788:00003130: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
789:00003140: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
790:00003150: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
791:00003160: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
792:00003170: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
793:00003180: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
794:00003190: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
795:000031a0: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
796:000031b0: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
797:000031c0: 4141 4141 4141 4141 4743 433a 2028 5562  AAAAAAAAGCC: (Ub
```

```
seed@cisc468:~/a3$ python3 -c "print(0x3100)"
12544
seed@cisc468:~/a3$ python3 -c "print(0x3020)"
12320
```

So now we know:

Prefix: beginning to 12352 (12320 + 32)

P/Q: 12352 to 12352 + 128 (since 128-byte hash block)

Middle: 12352 + 128 to 12544 + 32 (additional X is 32 byte into the block, Y needs to be too)

P: 12544 + 32 to 12544 + 32 + 128 (Y start + 32 is the start, and Y end is 128 bytes after that)

Suffix: 12544 + 32 + 128 to end

Here is the bash code that extracts Prefix, P, Q, Middle and Suffix and assembles them.

```bash
#!/bin/bash


X_START=12320
XOFFSET=12352 # This must be a multiple of 64
Y_START=12544


# Prefix: first 12352 bytes
head -c $XOFFSET task4 > prefix.bin


# Generate two collision blocks from the prefix with the same hash
./md5collgen -p prefix.bin -o col1.bin col2.bin
```

```
# Ensure current two binary files still have the same hash
md5sum col1.bin col2.bin

# Extract P and Q (we only need the 128-byte collision output)
tail -c 128 col1.bin > P.bin
tail -c 128 col2.bin > Q.bin

# Middle: 96 bytes (from 12480 to 12575)
# Because X was offset by 32 bytes, the middle section should cut 32 bytes into Y
MIDDLE_START=$((XOFFSET + 128))
MIDDLE_END=$((Y_START + 32))
MIDDLE_LEN=$((MIDDLE_END - MIDDLE_START))

tail -c +$((MIDDLE_START + 1)) task4 | head -c $MIDDLE_LEN > middle.bin

SUFFIX_START=$((MIDDLE_END + 128))
tail -c +$((SUFFIX_START + 1)) task4 > suffix.bin

# Assemble programs
# Benign = (prefix | P) | middle | P | suffix
cat col1.bin middle.bin P.bin suffix.bin > prog1
# Malicious = (prefix | Q) | middle | P | suffix
cat col2.bin middle.bin P.bin suffix.bin > prog2

chmod +x prog1 prog2
md5sum prog1 prog2

echo "Program 1 output (benign):"
./prog1
echo "Program 2 output (malicious):"
./prog2
```

Terminal output

```
seed@cisc468:~/a3$ ./task4.sh
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'col1.bin' and 'col2.bin'
Using prefixfile: 'prefix.bin'
Using initial value: f6a2f43109bea7a036fcffdd7c922d9a

Generating first block: ..........................................................
Generating second block: S11.....................
Running time: 70.0785 s
a82ed4bb140caca502a296ea3cba2221  col1.bin
a82ed4bb140caca502a296ea3cba2221  col2.bin
ba1905bb1ab47e6ce51b5feb94103a9a  prog1
ba1905bb1ab47e6ce51b5feb94103a9a  prog2
Program 1 output (benign):
Benign: running safe functionality.
Program 2 output (malicious):
Malicious: running dangerous functionality! >:)
```

We can see we have successfully produced two programs with the exact same hash, and one runs the benign functionality and one runs the malicious functionality!

These results show why the attack works and why it is dangerous. The two executables have the same MD5 hash because they share the same prefix and suffix, while the differing 128-byte collision regions were specially generated so that the internal MD5 state becomes identical after processing them. As a result, both complete files produce the same final hash even though some bytes inside the embedded arrays are different.

In the benign program, the inserted blocks make the contents of arrays X and Y identical, so the memcmp test succeeds and the safe branch is executed. In the malicious program, one of those regions is replaced with the alternate collision block, so X and Y are no longer equal, causing the program to follow the malicious branch instead. This demonstrates the real security problem with MD5 collisions: a hash value alone does not guarantee that two programs behave the same, which is why MD5 is not suitable for software integrity checking or digital signatures.